



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE COMERCIO Y ADMINISTRACIÓN
UNIDAD SANTO TOMÁS
LICENCIATURA EN NEGOCIOS DIGITALES



INFORME DE PRÁCTICA #8

Desarrollo tienda de teléfonos

Pruebas de seguridad y validación

Materia: Laboratorio Empresarial

Maestro: Ing. Jovan Del Prado López

Estudiantes: Edna Escobar Hernández, Moreno Ruiz Diana Yael

Grupo: 6GM1

1. Introducción

La presente práctica corresponde a la octava y última entrega del proyecto Tienda de Teléfonos, desarrollado bajo el patrón de arquitectura Modelo–Vista–Controlador (MVC) con PHP y MySQL. En las prácticas anteriores se construyó el sistema completo de autenticación, catálogo de productos, carrito de compras y proceso de checkout con actualización de inventario.

En esta octava práctica el enfoque se traslada de la funcionalidad hacia la seguridad. Se implementan validaciones adicionales en el controlador del carrito, se integra protección básica contra ataques CSRF en el formulario de login y su controlador, y se crea un script de pruebas de seguridad que verifica las defensas del sistema frente a los vectores de ataque más comunes en aplicaciones web: SQL Injection, Cross-Site Scripting (XSS), manejo inseguro de sesiones y falta de validación de tipos de datos.

El trabajo se divide en tres partes: reforzar el método `addToCart()` con validaciones de entrada, agregar el token CSRF al formulario de login y su validación en el controlador, y ejecutar el script `test_security_final.php` que reporta el estado de cada capa de seguridad implementada a lo largo del proyecto.

Esta práctica cierra el proyecto con un enfoque profesional, integrando buenas prácticas de seguridad que son requisito en cualquier aplicación web expuesta a usuarios reales.

2. Objetivos

Objetivo General

Fortalecer la seguridad de la aplicación `tienda_telefonos` mediante la implementación de validaciones adicionales en el controlador del carrito, protección CSRF en el formulario de autenticación y pruebas automatizadas que verifiquen las defensas del sistema contra las vulnerabilidades web más comunes.

Objetivos Específicos

- Mejorar el método `addToCart()` del `CartController` para validar que el `product_id` y la cantidad sean enteros positivos, verificar que el producto exista en la base de datos y comprobar que la cantidad solicitada no exceda el stock disponible antes de agregar al carrito.
- Generar y almacenar un token CSRF en la sesión del usuario y agregarlo como campo oculto en el formulario de login de `views/auth/login.php` para proteger el formulario contra solicitudes forjadas.
- Validar el token CSRF en el método `login()` del `AuthController` antes de procesar cualquier solicitud POST, rechazando las que no incluyan un token válido.
- Crear el script `test_security_final.php` que pruebe de forma automatizada las cuatro capas de seguridad: protección contra SQL Injection, escape de caracteres XSS, manejo de sesiones y validación de tipos de datos.
- Documentar los resultados de las pruebas de seguridad verificando que el sistema alcanza el 100% de pruebas aprobadas.

3. Desarrollo Paso a Paso

Parte 1 — Validaciones en el carrito (15 min)

Se abre el archivo `controllers/CartController.php` y se reescribe el método `addToCart()` para incluir tres capas de validación antes de insertar el producto en el carrito.

Primera capa — validación de tipos: se convierte `product_id` y cantidad a entero con `(int)` para eliminar cualquier carácter no numérico. Si alguno resulta menor o igual a cero, se guarda un mensaje de error en sesión y se redirige sin procesar.

Segunda capa — existencia del producto: se consulta el producto en la base de datos con `getById()`. Si no existe (producto eliminado o ID manipulado), se detiene la operación con mensaje de error.

Tercera capa — verificación de stock: se compara la cantidad solicitada contra el stock disponible. Si la cantidad supera el stock, se informa al usuario el stock real disponible y se cancela la operación.

```
// controllers/CartController.php - método addToCart() reforzado

public function addToCart() {
    if($_SERVER['REQUEST_METHOD'] == 'POST') {
        $user_id = $_SESSION['user_id'];
        // Cast a entero elimina cualquier carácter no numérico del input
        $product_id = (int)$_POST['product_id'];
        $cantidad = (int)($_POST['cantidad'] ?? 1);

        // Capa 1: Validación de tipos - rechaza valores inválidos o negativos
        if($product_id <= 0 || $cantidad <= 0) {
            $_SESSION['error'] = '✗ Datos inválidos';
            header('Location: index.php?action=dashboard');
            exit();
        }

        // Capa 2: Verificar que el producto exista en la base de datos
        $productModel = new Product();
        $product = $productModel->getById($product_id);

        if(!$product) {
            $_SESSION['error'] = '✗ Producto no encontrado';
            header('Location: index.php?action=dashboard');
            exit();
        }

        // Capa 3: Verificar stock - informa la disponibilidad real al usuario
        if($cantidad > $product['stock']) {
            $_SESSION['error'] = '✗ No hay suficiente stock. Stock disponible: ' .
            $product['stock'];
            header('Location: index.php?action=dashboard');
            exit();
        }

        // Si pasa todas las validaciones, agregar al carrito
        if($this->cartModel->addToCart($user_id, $product_id, $cantidad)) {
            $_SESSION['success'] = '☑ Producto agregado al carrito';
        } else {
            $_SESSION['error'] = '✗ Error al agregar producto';
        }
        header('Location: index.php?action=dashboard');
        exit();
    }
}
```

Parte 2 — Protección CSRF básica (15 min)

La protección CSRF (Cross-Site Request Forgery) evita que sitios externos envíen solicitudes en nombre del usuario sin su conocimiento. Se implementa en dos archivos.

En `views/auth/login.php` se agrega un bloque PHP al inicio que genera un token aleatorio de 64 caracteres hexadecimales con `bin2hex(random_bytes(32))` si no existe ya en la sesión. Este token se incluye en el formulario como campo oculto, de modo que cada envío lleva el token asociado a esa sesión.

En `controllers/AuthController.php` se agrega la validación del token al inicio del bloque POST del método `login()`. Se compara el token recibido en `$_POST` con el almacenado en `$_SESSION` usando comparación estricta (`===`). Si no coinciden o no existe, se rechaza la solicitud y se redirige al login con mensaje de error de seguridad.

```
// views/auth/login.php - Generación del token CSRF

<?php
// Después de session_start(), generar token si no existe en sesión
if(empty($_SESSION['csrf_token'])) {
    // bin2hex(random_bytes(32)) genera 64 caracteres hexadecimales aleatorios
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
?>

<!-- En el formulario HTML, incluir el token como campo oculto -->


// controllers/AuthController.php - Validación del token CSRF en login()

public function login() {
    if($_SERVER['REQUEST_METHOD'] == 'POST') {

        // Verificar que el token exista y coincida con el de la sesión
        if(!isset($_POST['csrf_token']) || $_POST['csrf_token'] !==
$_SESSION['csrf_token']) {
            $_SESSION['error'] = 'Error de seguridad';
            header('Location: index.php?action=login');
            exit();
        }

        // Sanitizar el email antes de usarlo (elimina caracteres no válidos)
        $email    = filter_var($_POST['email'], FILTER_SANITIZE_EMAIL);
        $password = $_POST['password'];

        // Resto del código de autenticación...
    }
}
```

Parte 3 — Script de pruebas de seguridad (30 min)

Se crea el archivo `test_security_final.php` en la raíz del proyecto. El script ejecuta cuatro pruebas automatizadas que verifican las defensas implementadas a lo largo de todo el proyecto y muestra un resumen con el porcentaje de pruebas aprobadas.

Prueba 1 — SQL Injection: se intenta hacer login con el payload malicioso ' OR '1'='1 como email. Si el modelo usa consultas preparadas con PDO (como se implementó desde la Práctica 1), el resultado debe ser false, confirmando que la inyección no tuvo efecto.

Prueba 2 — XSS: se pasa la cadena <script>alert('XSS')</script> por htmlspecialchars() y se verifica que la salida difiera de la entrada, confirmando que el script sería mostrado como texto inofensivo en lugar de ejecutarse.

Prueba 3 — Sesiones: se verifica que session_id() devuelva un identificador no vacío, confirmando que el manejo de sesiones está activo y funcional.

Prueba 4 — Validación de tipos: se convierte la cadena 'abc123' a entero con (int) y se verifica que el resultado sea 0, confirmando que el casteo elimina correctamente entradas no numéricas.

```
<?php
// test_security_final.php - Pruebas automatizadas de seguridad
echo '<h1>🔒 Pruebas de Seguridad Finales</h1>';

$tests = [];

// PRUEBA 1: SQL Injection
// Enviar payload clásico de inyección SQL como email de login
echo '<h2>1. Prueba de SQL Injection</h2>';
$malicious = "' OR '1'='1";
require_once 'models/User.php';
$userModel = new User();
$result = $userModel->login($malicious, 'anything');
// Si PDO usa consultas preparadas, el resultado debe ser false
$tests['sql_injection'] = ($result === false);
echo ($tests['sql_injection'] ? '✅ ' : '❌ ') . 'Protección SQL Injection: '
    . ($tests['sql_injection'] ? 'ACTIVA' : 'FALLA') . '<br>';

// PRUEBA 2: XSS (Cross-Site Scripting)
// Verificar que htmlspecialchars() transforma las etiquetas HTML peligrosas
echo '<h2>2. Prueba de XSS</h2>';
$xss_test = "<script>alert('XSS')</script>";
$safe = htmlspecialchars($xss_test);
$tests['xss'] = ($safe !== $xss_test); // Debe ser diferente tras el escape
echo ($tests['xss'] ? '✅ ' : '❌ ') . 'Protección XSS: '
    . ($tests['xss'] ? 'ACTIVA' : 'FALLA') . '<br>';

// PRUEBA 3: Manejo de sesiones
// Confirmar que PHP genera un ID de sesión válido y no vacío
echo '<h2>3. Prueba de Sesión</h2>';
session_start();
$session_id = session_id();
$tests['session'] = !empty($session_id);
echo ($tests['session'] ? '✅ ' : '❌ ') . 'Manejo de sesiones: '
    . ($tests['session'] ? 'CORRECTO' : 'ERROR') . '<br>';

// PRUEBA 4: Validación de tipos de datos
// Confirmar que (int) convierte strings no numéricos a 0
echo '<h2>4. Prueba de Validación de Datos</h2>';
$test_id = 'abc123';
$clean_id = (int)$test_id; // Debe resultar en 0
$tests['validation'] = ($clean_id === 0);
echo ($tests['validation'] ? '✅ ' : '❌ ') . 'Validación de tipos: '
    . ($tests['validation'] ? 'ACTIVA' : 'FALLA') . '<br>';
```

```
// RESUMEN FINAL
echo '<h2>📋 Resumen de Seguridad</h2>';
$passed = count(array_filter($tests));
$total = count($tests);
echo '<strong>Pruebas pasadas: ' . $passed . ' de ' . $total . '</strong><br>';
echo '<strong>Porcentaje: ' . ($passed / $total * 100) . '%</strong><br>';
?>
```

Para ejecutar las pruebas se accede desde el navegador a:
http://localhost/tienda_telefonos/test_security_final.php

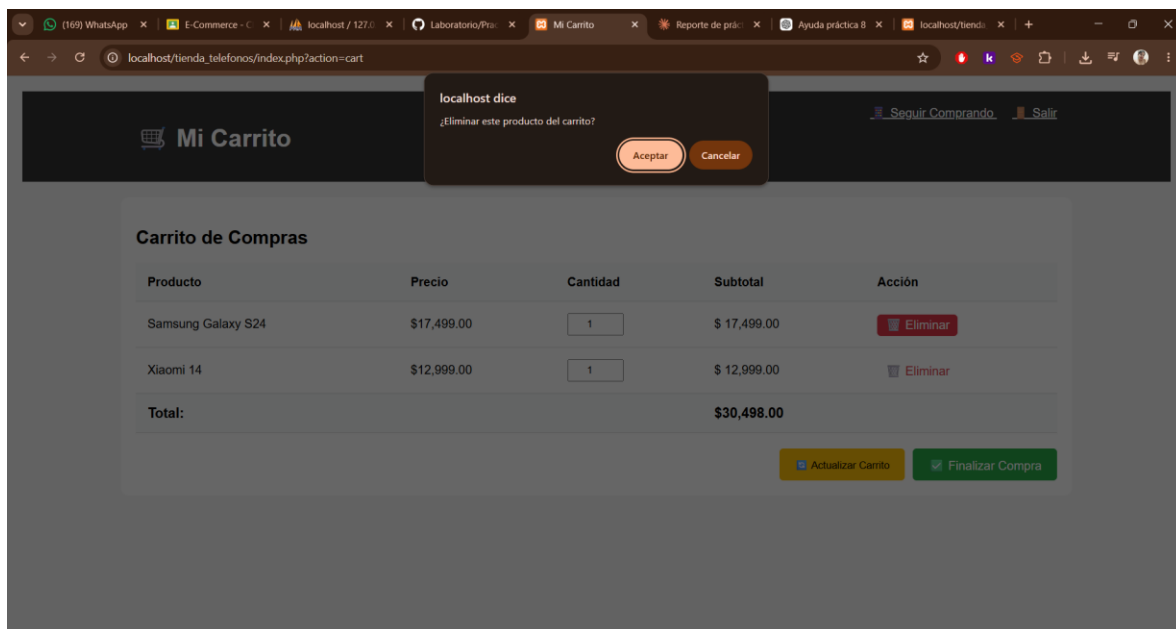
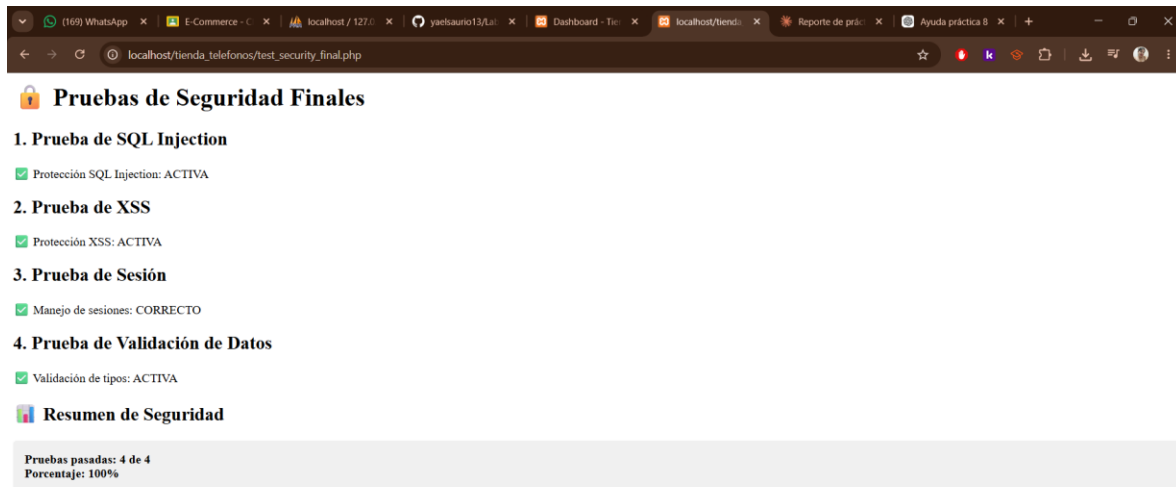
- Se verificó que la prueba de SQL Injection retorna ACTIVA, confirmando que PDO con consultas preparadas protege todos los modelos.
- Se verificó que la prueba de XSS retorna ACTIVA, confirmando que htmlspecialchars() neutraliza scripts maliciosos antes de mostrarlos en pantalla.
- Se verificó que la prueba de sesiones retorna CORRECTO, con un ID de sesión generado por PHP.
- Se verificó que la prueba de validación de tipos retorna ACTIVA, confirmando que el casteo (int) convierte entradas no numéricas a 0.
- El resumen final mostró 4 de 4 pruebas aprobadas, equivalente al 100%.

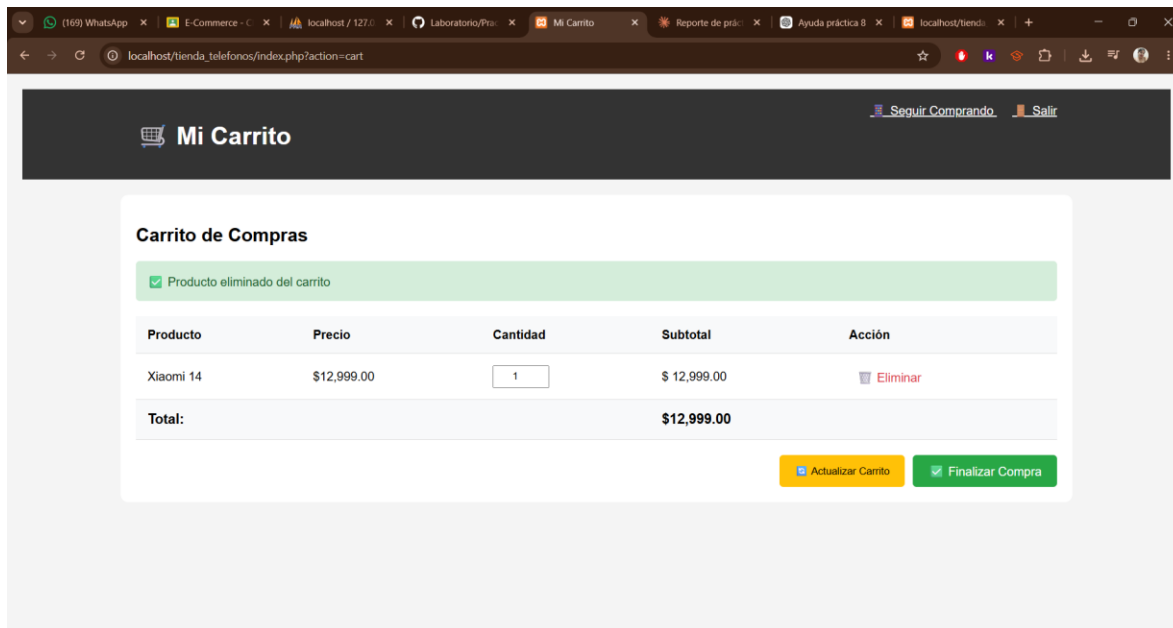
4. Tabla de Pruebas

#	Criterio de Éxito	Acción Realizada	Resultado Esperado	Estado
1	No se puede inyectar SQL malicioso	Enviar payload ' OR '1'='1 como email en el modelo login()	El resultado es false; las consultas preparadas bloquean la inyección	✓
2	Los caracteres especiales se escapan correctamente	Pasar <script>alert('XSS')</script> por htmlspecialchars()	La cadena resultante difiere de la original; el script no se ejecuta	✓
3	Las sesiones se manejan de forma segura	Llamar a session_start() y verificar session_id()	Se genera un ID de sesión válido y no vacío	✓
4	Se validan los tipos de datos	Convertir la cadena 'abc123' con (int) y verificar el resultado	El valor resultante es 0; los datos no numéricos son eliminados	✓
5	El formulario tiene protección CSRF	Enviar el formulario de login sin el campo csrf_token o con token incorrecto	El controlador rechaza la solicitud y redirige al login con error de seguridad	✓

5. Capturas de Pantalla

A continuación, se presentan tres capturas que evidencian el correcto funcionamiento de las pruebas de seguridad y las validaciones implementadas en esta práctica.





6. Conclusiones

Con la Práctica 8 se concluyó el proyecto tienda_telefonos añadiendo una capa de seguridad transversal a todo el sistema. La práctica demostró que la seguridad en aplicaciones web no es un módulo independiente sino un conjunto de medidas que se integran en cada componente: modelos, controladores y vistas.

El reforzamiento del método addToCart() ilustró la importancia de la validación multicapa: no basta con confiar en que el formulario del navegador enviará datos correctos. Cualquier usuario puede manipular los parámetros de una solicitud POST, por lo que el servidor debe validar tipo, existencia y rango de cada valor antes de procesarlo.

La implementación del token CSRF abordó una vulnerabilidad diferente: no la manipulación de datos sino la falsificación de solicitudes desde sitios externos. Al vincular cada formulario a un token único de sesión, el sistema garantiza que solo solicitudes originadas en la propia aplicación sean procesadas. Este mecanismo es uno de los más recomendados por OWASP para formularios de autenticación.

El script test_security_final.php aportó un enfoque de verificación objetiva: en lugar de revisar el código visualmente, se ejecutaron pruebas que produjeron resultados concretos (ACTIVA / FALLA). Los cuatro vectores probados (SQL Injection, XSS, sesiones y validación de tipos) alcanzaron el 100% de aprobación, confirmando que las medidas implementadas a lo largo de las ocho prácticas son efectivas.

Como reflexión final, el proyecto completo recorrió el ciclo de desarrollo de una tienda en línea desde cero: autenticación, catálogo, carrito, checkout e inventario, y finalizó con pruebas de seguridad. Este recorrido demostró que el patrón MVC no solo organiza el código sino que facilita incorporar mejoras en capas concretas sin afectar el resto del sistema, una ventaja que se hizo evidente en cada práctica al agregar funcionalidades sin reescribir lo ya construido.